Project Title: MicroGoals

Team ID: SWTID-2026-2521

Team Leader: SRI SAVITHA.M

Team member: SUMITHRA.S

Team member: SHOBIKA.V

Team member: SUMITHA.M

Abstract

MicroGoals is a secure backend RESTful API designed to help users create, manage, and track their personal goals. The system is developed using Node.js, Express.js, and MongoDB. It allows users to register, log in, and perform CRUD operations on their goals. Authentication is implemented using JWT (JSON Web Token) to ensure that only authorized users can access their data. The application provides a structured way for efficiently.

Introduction

Goal setting is an individual to organize their goals and monitor their progress important part of personal productivity and self-development. MicroGoals provides a digital platform where users can securely manage their goals. The system allows users to: Register and create an account, Log in securely, Add, update, view, and delete personal goals. By using a backend API architecture, MicroGoals ensures scalability, security, and efficient data management.

Problem Statement

Many individuals want to track their goals but lack a simple and secure system to manage them. Traditional methods like notebooks or simple notes apps do not provide structured goal management. Existing applications may be complex or include unnecessary features. Therefore, there is a need for a lightweight backend

system that allows users to easily create and manage their goals with proper authentication and data security.

Project Overview

MicroGoals is a secure backend RESTful API developed using Node.js, Express.js, and MongoDB. It enables users to register, log in, and manage their personal goals through CRUD operations. The system uses JWT authentication and middleware to ensure data security and user-specific access control.

The system includes:

- User Registration & Login
- JWT-based Authentication
- CRUD Operations for Goals
- Protected Routes
- Centralized Error Handling

Project Instruction

- The project follows REST API principles.
- Authentication is handled using JWT.
- MongoDB is used as the database.
- APIs are tested using Thunder Client.
- The project follows MVC architecture.

Pre-requisites

Before running the project, the following tools are required:

- Node.js (v18 or higher)
- Express.js
- MongoDB (Local / Atlas)
- VS Code
- Thunder Client (Extension)
- Basic knowledge of JavaScript & REST APIs

Objectives

Provide secure user registration and login.

Allow users to create, view, update, and delete goals.

Protect user data using JWT authentication.

Store user and goal data in MongoDB database.

Ensure that users can access only their own goals.

Build a simple and scalable backend system. FEATURES

USERS FLOW

1. User registers

2. User logs in

3. JWT token generated

4. User sends token in header

5. Access protected routes

6. Perform CRUD operations

PROJECT SETUP AND CONFIGURATION

◇ Folder Structure

```
MicroGoals/
├── middleware/
│   ├── authMiddleware.js
│   └── errorMiddleware.js
├──Schema/
├──User.js
├──GoalSchema.js
├── models/
│   ├── userModel.js
│   └── goalModel.js
│
├── routes/
│   ├── authRoutes.js
│   └── goalRoutes.js
│
├──. env
├── Index.js
└── package. Json
```

◇ Project Setup

1. nm. in it -y

2. Install dependencies:

nm. install express mongoose honeytoken crypts doting

3. Start server:

node server.js

BACKEND DEVELOPMENT

- Created Express Server

An Express server was set up to handle API requests, connect routes, and enable JSON parsing for backend operations

- Connected MongoDB

- Implemented Authentication APIs

User registration and login APIs were created with password hashing and JWT token generation for secure authentication.

- Implemented Goal CRUD APIs

Goal CRUD APIs allow authenticated users to Create, Read, Update, and Delete their personal goals. These APIs follow RESTful architecture to manage data in a structured and secure way.

- The **Create API** allows users to add new goals to the database. Each goal is linked to a specific user using an **ushered**, ensuring that goals belong only to that user.
- The **Read API** allows authenticated users to retrieve only their own goals. The system filters data using the authenticated user's ID to prevent unauthorized access.

- The **Update API** allows users to modify their existing goals. Validation checks ensure that only the owner of the goal can update it.
- The **Delete API** allows users to permanently remove their goals from the system. Security checks make sure users cannot delete goals belonging to others.
- All CRUD routes are protected using **Authentication Middleware**, which verifies the JWT token before allowing access. If the token is missing or invalid, the system returns a **401 Unauthorized** error.
- An **Error Middleware** is also implemented to handle errors centrally. It captures runtime and validation errors and returns consistent JSON responses with proper HTTP status codes like **200, 201, 400, 401, and 404**.
- This implementation ensures secure data handling, proper authorization, and reliable API functionality for goal management.

```
const express = require("express");
const router = express. Router ();
const goal Model = require (“. /Model/Goal Model");
const AUTH middleware = require (“. /Middleware’s/AUTH Middleware");

router. Post ("/create", AUTH middleware, async (req, res, next) => {
try {
const {title} = rebody;
const goal = new goal Model ({
title: title,
user: request,
});
await goal. Save ();
restates (200). Json ({
success: true,
data: goal,
});
} catch (err) {
return next(err);
}
});
```

```
router. Get ("/:id", AUTH middleware, async (req, res, next) => {
try {
const id = req.params.id;
const data = await goalModel.findOne({
_id: id,
user: req.user,
});
res.status(200).json({
success: true,
data: data,
});
} catch (error) {
return next(error);
}
});

router. Put ("/:id", AUTH middleware, async (req, res, next) => {
try {
const goal = await goalModel.findOneAndUpdate(
{_id: req.params.id, user: request},
{ completed: req.body.completed },
{new: true},
);
restates (200). Json ({
success: true,
message: "the goal is completed",
});
} catch (error) {
return next(error);
}
});

router. Delete ("/:id", AUTH middleware, async (req, res, next) => {
try {
await goalModel.findOneAndDelete({
_id: req.params.id,
});
```

```
restates (200). Json ({message: "Goal deleted”});
} catch (error) {
next(error);
}
});

module. Exports = router;
```

- Added Auth Middleware
- Auth middleware was implemented to verify JWT tokens and secure protected routes from unauthorized access.
- Added Error Middleware

An error middleware was added to handle application errors centrally and return proper status codes and messages.

DATABASE DEVELOPMENT

CONFIGURE MongoDB

- Use MongoDB Atlas or Local MongoDB
- Store connection string in. env

CREATE DATABASE CONNECTION

Index.js file:

- Use mongoose. Connect ()

```
//mongoDB connection
mongoose
  .connect(Mongo_Url)
  .then(() => console.log("MongoDB Connected"))
  .catch((err) => console.log(err));
```

CREATE SCHEMA AND MODELS

- User Schema

The User Schema defines the user data structure with name, email (unique), and hashed password fields.

```
JS User.js
const mongoose = require("mongoose");
const Schema = mongoose.Schema;
const userSchema = new Schema({
  name: {
    type: String,
    required: true,
  },
  email: {
    type: String,
    unique: true,
  },
  password: {
    type: String,
    required: true,
  },
});
module.exports=userSchema;
```

- Goal Schema (with ushered reference)

The Goal Schema stores goal details and includes an ushered reference to associate each goal with a specific user.

PROJECT EXECUTION

Project Execution Steps

1. Start MongoDB
2. Run server
3. Test APIs in Thunder Client
4. Register User
5. Login and get JWT
6. Perform CRUD operations

Conclusion

The MicroGoals API project demonstrates how to build a secure backend application using Node.js, Express.js, and MongoDB. The system allows users to register, log in, and manage their personal goals through CRUD operations. By implementing JWT authentication and protected routes, the application ensures that only authorized users can access and modify their data. This project helps in understanding important backend development concepts such as RESTful API design, database integration, authentication, middleware usage, and API testing using Thunder Client